Documento Tecnico IANUS

Architettura di Integrazione e Modello Dati

Antonio Russo - 9 dicembre 2025

1. Introduzione

Il presente documento descrive l'architettura di integrazione, il modello dati e le modalità operative si IANUS progettata per acquisire informazioni da sistemi esterni di Network Management (NMS).
L'obiettivo principale è garantire la normalizzazione dei dati provenienti da piattaforme eterogenee, fornendo un modello sintattico uniforme che possa essere elaborato dagli algoritmi di analisi dell'appliance.

2. Architettura di Integrazione

2.1 Approccio a Plugin OpenAPI

L'integrazione con sistemi di terze parti è implementata tramite un'architettura **plugin-based**, conforme a interfacce OpenAPI.
Ogni sistema NMS supportato è gestito tramite un plugin dedicato che ha le seguenti responsabilità:

- Effettuare richieste verso il sistema esterno (modalità **pull**) oppure ricevere dati in ingresso (modalità **push**).
- Convertire i dati ottenuti nel **formato dati standard IANUS**.
- Garantire l'isolamento delle logiche specifiche del singolo NMS rispetto all'API unificata di IANUS.

2.2 Sistemi Supportati

L'architettura attualmente include lo sviluppo di plugin per i seguenti sistemi di monitoraggio:

- **OpenNMS**
- **Zabbix**
- **Nagios Core/Icinga 2**
- **Prometheus**
- **Syslog/Apache Nifi/ Elastic Search**

L'estensibilità dell'architettura consente di integrare ulteriori sistemi seguendo lo stesso modello.

3. Modello dei Dati di Performance

3.1 Struttura dei Dati di Performance

I dati di performance rappresentano misurazioni ripetute nel tempo e sono acquisiti periodicamente dai sistemi esterni.

Il formato dati richiesto dall'appliance IA è il seguente:

`Timestamp : metric : instance : value`

- **Timestamp**: epoca UNIX o formato temporale coerente, indicante il momento della misurazione.
- **Metric**: nome della metrica (es. CPU, Traffic).
- **Instance**: identificatore dell'oggetto monitorato (es. indice CPU, interfaccia, tuple IP).
- **Value**: valore della misurazione numerica o testuale.

3.2 Esempi di Metriche Semplici

Esempi di metriche elementari:

- `1765042754:"CPU":1:40` → Utilizzo CPU core 1 = 40%
- `1765042754:"CPU":2:50` → Utilizzo CPU core 2 = 50%
- `1765042754:"Traffic":"Gigabit0/0":200` → Traffico su interfaccia Gigabit0/0 = 200 MB/s

3.3 Metriche Complesse (es. NetFlow)

Le metriche provenienti da sistemi di flow analysis possono presentare istanze complesse, come tuple IP:porta.

Esempi:

- `1765042754:"Numero di Connessioni TCP in uscita":"192.168.0.1:222-212.151.3.2:446":1`
- `1765042754:"Traffic":"192.168.0.1:222-212.151.3.2:446":10`
- `1765042754:"Programma":"192.168.0.1:222-212.151.3.2:446":"/hacker/trojan"`
- `1765042754:"PID":"192.168.0.1:222-212.151.3.2:446":1234`

Questo approccio consente di acquisire e rappresentare qualunque metrica, indipendentemente dalla complessità o dalla semantica associata dall'NMS di origine.

4. Modello dei Dati di Evento

4.1 Struttura degli Eventi

Gli eventi rappresentano occorrenze puntuali, non periodiche, originate dai sistemi esterni.

Il formato previsto è:

```
Timestamp : event_id : instance : log_level
```

- **Timestamp**: istante in cui l'evento è stato rilevato.
- **Event_id**: identificatore dell'evento (codice univoco o descrizione).
- **Instance**: elemento o oggetto a cui l'evento si riferisce (host, servizio, interfaccia, ecc.).
- **Log_level**: livello di severità (es. INFO, WARNING, CRITICAL).

4.2 Tipologie di Eventi Supportate

L'appliance può ricevere e gestire eventi provenienti da differenti sorgenti, tra cui:

- **SYSLOG**
- **SNMP Trap**
- **Eventi di Fault Management** generati dagli NMS
- **Eventi di sistema Windows**
- Altre sorgenti strutturate o semi-strutturate

5. Modello dei Dati Sintetico - Struttura LOG di sistema

Il sistema è configurato per la gestione di log strutturati secondo un insieme di campi standardizzati. Ogni record di log rappresenta un evento o uno stato rilevato su un server monitorato.

5.1 Struttura dei Campi del Log

- **`timestamp`**
  Indica la data e l'ora in cui l'evento è stato rilevato. Può essere espresso in formato epoch o ISO8601, a seconda della configurazione del sistema.
- **`server_id`**
  Identificatore univoco del server o del nodo che ha generato il log. Consente la correlazione degli eventi tra sistemi diversi.

- **`cpu_usage`**
  Percentuale di utilizzo della CPU al momento del log. Valore numerico compreso tra 0 e 100.

- **`memory_usage`**
  Percentuale di memoria utilizzata dal sistema nel momento rilevato. Valore numerico compreso tra 0 e 100.

- **`disk_io`**
  Quantità di operazioni di I/O disco eseguite nell'intervallo di riferimento. Può rappresentare letture, scritture o un aggregato delle due.

- **`network_traffic`**
  Volume di traffico di rete espresso in MB nel periodo osservato. Utilizzato per analizzare il carico della rete e lo stato delle interfacce.

- **`error_count`**
  Numero di errori rilevati nel periodo di osservazione. Può includere errori di sistema, applicativi o hardware a seconda delle configurazioni di raccolta.

- **`log_level`**
  Livello di severità associato al log. Può assumere uno dei seguenti valori standard:

  - `INFO` – Informazione generale
  - `WARNING` – Condizione anomala non critica
  - `ERROR` – Errore rilevante che richiede attenzione
  - `CRITICAL` – Condizione grave che impatta la continuità del servizio

6. Principi del Modello Sintattico

Il sistema utilizza un modello **puramente sintattico**, non basato su ontologie o semantiche proprietarie dei sistemi di monitoraggio.

Criteri di disegno principali

- Uniformazione dei dati provenienti da fonti diverse.
- Capacità di rappresentare qualunque metrica o evento senza restrizioni di formato.
- Elevata estensibilità con integrazione di nuove metriche e sistemi.
- Facilità di elaborazione tramite algoritmi di IA, che possono attribuire pesi o importanza alle metriche senza dipendenze semantiche.

7. Conclusioni

L'architettura a plugin con modello dati sintattico consente a IANUS di interoperare con sistemi di monitoraggio eterogenei, normalizzando metriche ed eventi in un formato uniforme e facilmente processabile.
Questo approccio garantisce flessibilità, scalabilità e capacità di integrazione con sistemi presenti e futuri, fornendo una base solida per analisi avanzate e applicazioni di intelligenza artificiale.